L_r for the redistributed reward, such that $L_v = L_o + L_r$ `quality`. Parameter values were taken from the original paper [115] and implementation [21]. Additionally, a coarse hyperparameter search was performed with value function coefficients $\{0.1, 1, 10\}$ and replacing the static entropy coefficient by a Proportional Control scaling of the entropy coefficient. The Proportional Control target entropy was linearly decreased from 1 to 0 over the course of training. PPO baseline hyperparamters were used for PPO with RUDDER without changes.
Parameter values are listed in Table A5.

Reward redistribution model. The loss of the reward redistribution model for a sample is composed of four parts. (i) The main loss L_m, which is the squared prediction loss of $\mathbf{g}$ at the last time step T of the episode

$$L_m \;=\; (\mathbf{g} \;-\; \widehat{\mathbf{g}}_T)^2 \;, \tag{A268}$$

(ii) the continuous prediction loss L_c of $\mathbf{g}$ at each time step

$$L_c \;=\; \frac{1}{T+1} \sum_{t=0}^{T} (\mathbf{g} \;-\; \widehat{\mathbf{g}}_t)^2 \;, \tag{A269}$$

(iii) the loss L_e of the prediction of the output at $t+10$ at each time step t

$$L_e \;=\; \frac{1}{T-9} \sum_{t=0}^{T-10} \left(\widehat{\mathbf{g}}_{t+10} \;-\; (\widehat{\mathbf{g}_{t+10}})_t \right)^2 \;, \tag{A270}$$

as well as (iv) the loss on 3 auxiliary tasks. At every time step t, these auxiliary tasks are (1) the prediction of the action-value function $\widehat{q}_t$, (2) the prediction of the accumulated original reward $\tilde{r}$ in the next 10 frames $\sum_{i=t}^{t+10} \tilde{r}_i$, and (3) the prediction of the accumulated reward in the next 50 frames $\sum_{i=t}^{t+50} \tilde{r}_i$, resulting in the final auxiliary loss L_a as

$$L_{a1} \;=\; \frac{1}{T+1} \sum_{t=0}^{T} (q_t \;-\; \widehat{q}_t)^2 \;, \tag{A271}$$

$$L_{a2} \;=\; \frac{1}{T-9} \sum_{t=0}^{T-10} \left(\sum_{i=t}^{t+10} \tilde{r}_i \;-\; \left(\widehat{\sum_{i=t}^{t+10} \tilde{r}_i} \right)_t \right)^2 \;, \tag{A272}$$

$$L_{a3} \;=\; \frac{1}{T-49} \sum_{t=0}^{T-50} \left(\sum_{i=t}^{t+50} \tilde{r}_i \;-\; \left(\widehat{\sum_{i=t}^{t+50} \tilde{r}_i} \right)_t \right)^2 \;, \tag{A273}$$

$$L_a \;=\; \frac{1}{3} \; (L_{a1} \;+\; L_{a2} \;+\; L_{a3}) \;. \tag{A274}$$

The final loss for the reward redistribution model is then computed as

$$L \;=\; L_m \;+\; \frac{1}{10} \; (L_c \;+\; L_e \;+\; L_a) \;. \tag{A275}$$

The continuous prediction and earlier prediction losses L_c and L_e push the reward redistribution model toward performing an optimal reward redistribution. This is because important events that are redundantly encoded in later states are stored as early as possible. Furthermore, the auxiliary loss L_a speeds up learning by adding more information about the original immediate rewards to the updates.
The reward redistribution model is only trained on the lessons buffer. Training epochs on the lessons buffer are performed every 10^4 PPO updates or if a new sample was added to the lessons buffer. For each such training epoch, 8 samples are sampled from the lessons buffer. Training epochs are repeated until the reward redistribution quality is sufficient (`quality` > 0) for all replayed samples in the last 5 training epochs.
The reward redistribution model is not trained or used until the lessons buffer contains at least 32 samples and samples with different return have been encountered.
Parameter values are listed in Table A5.

PPO		RUDDER	
learning rate	$2.5 \cdot 10^{-4}$	learning rate	10^{-4}
policy coefficient	1.0	L_2 weight decay	10^{-7}
initial entropy coefficient	0.01	gradient clipping	0.5
value function coefficient	1.0	optimization	ADAM

Table A5: Left: Update parameters for PPO model. Entropy coefficient is scaled via Proportional Control with the target entropy linearly annealed from 1 to 0 over the course of learning. Unless stated otherwise, default parameters of *ppo2* implementation [21] are used. Right: Update parameters for reward redistribution model of RUDDER.

Sequence chunking and Truncated Backpropagation Through Time (TBPTT). Ideally, RUDDER would be trained on completed game sequences, to consequently redistribute the reward within a completed game. To shorten computational time for learning the reward redistribution model, the model is not trained on completed game sequences but on sequence chunks consisting of 512 time steps. The beginning of such a chunk is treated as beginning of a new episode for the model and ends of episodes within this chunk reset the state of the LSTM, so as to not redistribute rewards between episodes. To allow for updates on sequence chunks even if the game sequence is not completed, the PPO value function prediction is used to estimate the expected future reward at the end of the chunk. Utilizing TBPTT to further speed up LSTM learning, gradients for the reward redistribution LSTM are cut after every 128 time steps.

A4.2.4 Exploration

Safe exploration to increase the likelihood of observing delayed rewards is an important feature of RUDDER. We use a safe exploration strategy, which is realized by normalizing the output of the policy network to range $[0, 1]$ and randomly picking one of the actions that is above a threshold θ. Safe exploration is activated once per sequence at a random sequence position for a random duration between 0 and the average game length $\bar{l}$. Thereby we encourage long but safe off-policy trajectories within parts of the game sequences. Only 2 of the 8 parallel actors use safe exploration with $\theta_1 = 0.001$ and $\theta_1 = 0.5$, respectively. All actors sample from the softmax policy output.
To avoid policy lag during safe exploration transitions, we use those transitions only to update the reward redistribution model but not the PPO model.

A4.2.5 Results

Training curves for 3 random seeds for PPO baseline and PPO with RUDDER are shown in Figure A8 and scores are listed in Table A6 for all 52 Atari games. Training was conducted over 200M game frames (including skipped frames), as described in the experiments section of the main paper.
We investigated failures and successes of RUDDER in different Atari games. RUDDER failures were observed to be mostly due to LSTM failures and comprise e.g. slow learning in Breakout, explaining away in Double Dunk, spurious redistributed rewards in Hero, overfitting to the first levels in Qbert, and exploration problems in MontezumaRevenge. RUDDER successes were observed to be mostly due to redistributing rewards to important key actions that would otherwise not receive reward, such as moving towards the built igloo in Frostbite, diving up for refilling oxygen in Seaquest, moving towards the treasure chest in Venture, and shooting at the shield of the enemy boss UFO, thereby removing its shield.

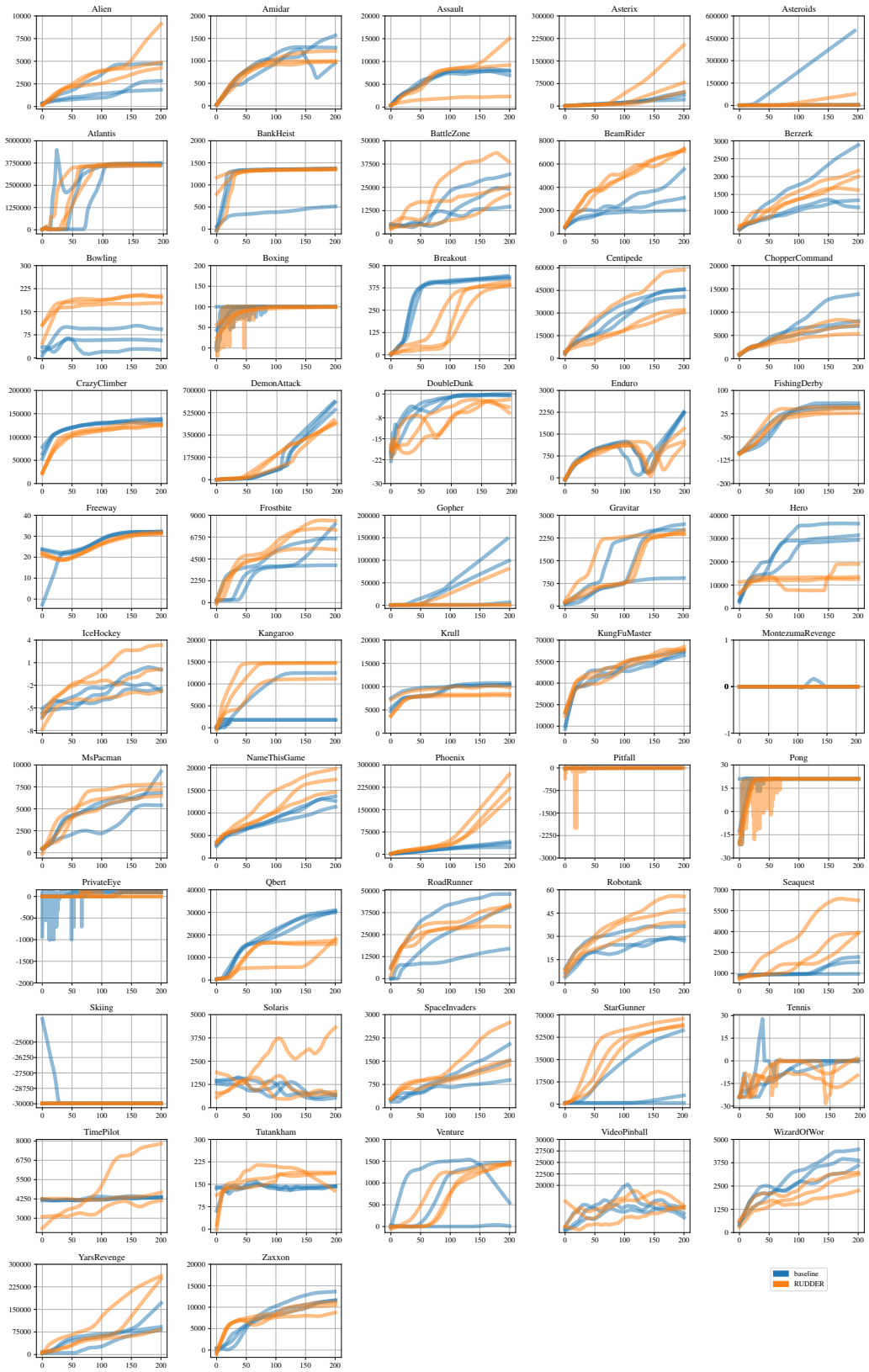

Figure A8: Training curves for PPO baseline and PPO with RUDDER over 200M game frames, 3 runs with different random seeds each. Curves show scores during training of a single agent that does not use safe exploration, smoothed using Locally Weighted Scatterplot Smoothing (y-value estimate using 20% of data with 10 residual-based re-weightings).